

Got it  — you want to parse JSON in **C#** using `System.Text.Json` where a property can be:

1. A **JSON array of objects**
2. Or an **empty string** (`""`), which should be treated as an **empty array**

This is not supported out-of-the-box, but we can solve it with a **custom** `JsonConverter` .

Complete Runnable Example

```
using System;
using System.Collections.Generic;
using System.Text.Json;
using System.Text.Json.Serialization;

// Model representing each item in the array
public class MyItem
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
}

// Custom converter to handle array OR empty string
public class FlexibleArrayConverter<T> : JsonConverter<List<T>>
{
    public override List<T> Read(ref Utf8JsonReader reader, Type typeToConvert, JsonSerializerOptions options)
    {

```

```
// Handle null
if (reader.TokenType == JsonTokenType.Null)
    return new List<T>();

// Handle empty string as empty array
if (reader.TokenType == JsonTokenType.String)
{
    string str = reader.GetString();
    return string.IsNullOrEmpty(str) ? new List<T>() : throw new JsonException("Expected
empty string or array.");
}

// Handle normal JSON array
if (reader.TokenType == JsonTokenType.StartArray)
{
    var list = new List<T>();
    while (reader.Read())
    {
        if (reader.TokenType == JsonTokenType.EndArray)
            return list;

        // Deserialize each element
        T element = JsonSerializer.Deserialize<T>(ref reader, options);
        list.Add(element);
    }
}
```

```
        throw new JsonException("Invalid JSON format for array property.");
    }

    public override void Write(Utf8JsonWriter writer, List<T> value, JsonSerializerOptions options)
    {
        JsonSerializer.Serialize(writer, value, options);
    }
}

// Root object containing the array property
public class MyData
{
    [JsonConverter(typeof(FlexibleArrayConverter<MyItem>))]
    public List<MyItem> Items { get; set; } = new();
}

class Program
{
    static void Main()
    {
        var options = new JsonSerializerOptions
        {
            PropertyNameCaseInsensitive = true,
            WriteIndented = true
        };
    }
}
```

```
// Example JSON with array
string jsonWithArray = @"{ ""Items"": [ { ""Id"": 1, ""Name"": ""A"" }, { ""Id"": 2,
""Name"": ""B"" } ] }";

// Example JSON with empty string
string jsonWithEmptyString = @"{ ""Items"": """" }";

// Parse both
var data1 = JsonSerializer.Deserialize<MyData>(jsonWithArray, options);
var data2 = JsonSerializer.Deserialize<MyData>(jsonWithEmptyString, options);

Console.WriteLine("Array case:");
Console.WriteLine(JsonSerializer.Serialize(data1, options));

Console.WriteLine("\nEmpty string case:");
Console.WriteLine(JsonSerializer.Serialize(data2, options));
}
}
```

How It Works

1. `FlexibleArrayConverter<T>` :
 1. If the JSON token is a **string** and empty → returns an empty list.

2. If the token is an **array** → deserializes normally.
3. If `null` → returns an empty list.
4. Otherwise → throws a `JsonException`.

2. Usage:

1. Apply `[JsonConverter(typeof(FlexibleArrayConverter<MyItem>))]` to the property.

3. Output Example:

```
Array case:
{
  "Items": [
    {
      "Id": 1,
      "Name": "A"
    },
    {
      "Id": 2,
      "Name": "B"
    }
  ]
}
```

```
Empty string case:
{
  "Items": []
}
```

If you want, I can also make a **version that works for** `IEnumerable<T>` **or arrays** instead of `List<T>` so it's more flexible.
Do you want me to prepare that version?